Type Checking in Lean

Type Theory and Formal Verification: A Comprehensive Overview

Academic Presentation

March 11, 2026

Study of Lean 4 Architecture

Outline

1. The Epistemological Foundation
2. A Crash Course in Type Theory
3. Dependent Type Theory (DTT)
4. The Curry-Howard Isomorphism
5. Lean's Pipeline Architecture
6. Machinery of Reduction

1. The Epistemological Foundation

Empirical Testing vs. Formal Verification

- **Software Testing**: Provides a localized guarantee. Demonstrates the absence of failure under specific, finite conditions.
- **Limitation**: Intrinsically limited by the impossibility of exploring an infinite state space.
- **Formal Verification**: Leverages mathematical proofs to guarantee the absolute impossibility of failure across all defined states.
- Correctness becomes a mathematically proven theorem rather than a statistical likelihood.

The Lean 4 Vanguard

- **Lean 4**: An interactive theorem prover and functional programming language.
- Developed by researchers at Microsoft Research and Carnegie Mellon University.
- Fundamentally erases the boundary between a theorem prover and a general-purpose programming language.
- Compiles to highly efficient C code safely.

Compile-Time Type Inhabitation

- In dynamically typed languages (like Python), $2 + 3 = 5$ evaluates to a runtime boolean `True`.
- In Lean, $2 + 3 = 5$ is treated as a complex *compile-time type inhabitation problem*.
- Understanding this divergence requires traversing type theory, the Curry-Howard isomorphism, and internal reduction algorithms.

2. A Crash Course in Type Theory

The Untyped λ-Calculus

- **Untyped λ-calculus**: Introduced by Alonzo Church (1930s).
- Comprises variables (x), function abstraction ($\lambda x.t$), and application ($t_1 t_2$).
- Turing-complete, but allows unrestricted application.
- **The Problem**: Evaluating a function upon itself can lead to paradoxical, non-terminating terms (mirroring naive set theory inconsistencies).

Simply Typed λ-Calculus ($\lambda^{\rightarrow}$)

- To introduce discipline and guarantee termination, Church formalized the simply typed lambda calculus.
- Every term possesses an intrinsic, immutable type.
- Function applications are strictly validated against these types.
- If $f : A \rightarrow B$ and $x : A$, then $f(x) : B$. Otherwise, it fails immediately.

Typing Judgments and Inference Rules

- **Typing Judgment**: $\Gamma \vdash t : A$
- "Under context Γ, term t possesses type A."
- A formal type theory defines mathematical objects via four rules:
  1. **Formation**: How to construct new types.
  2. **Introduction**: How to construct (inhabit) terms of a new type.
  3. **Elimination**: How to logically use or consume a term.
  4. **Computation**: How elimination interacts with introduction (reduction).

3. Dependent Type Theory (DTT)

The Leap to Dependent Types

- Simply typed systems are insufficient for rigorous mathematical reasoning.
- **Dependent Type Theory (DTT)**: Types can dynamically depend on runtime or compile-time values.
- **Example**: A generic `List` vs. a `Vector` α `n`. The type depends on the specific variable *n* of type `Nat`, allowing strict array bounds checking at compile time!

Paramount Type Constructors in Lean

- **Dependent Product (Π type)**:
  - Generalizes function types $A \to B$. The return type B depends on input value x.
  - Formally: $\Pi(x : A), B(x)$.
  - Directly translates to Universal Quantification ($\forall x : A, P(x)$).
- **Dependent Sum (Σ type)**:
  - Generalizes tuple types $A \times B$. Type of 2nd element depends on 1st element.
  - Directly translates to Existential Quantification ($\exists x : A, P(x)$).

Universes and Paradoxes

- If types behave like standard terms, do they have types themselves?
- Allowing `Type : Type` introduces Girard's Paradox (fatal logical inconsistency).
- Lean implements a strict, countably infinite hierarchy of universes: **Calculus of Inductive Constructions (CIC)**.
- `Prop (Sort 0)`: Logical propositions.
- `Type 0 (Sort 1)`: Data types (Nat, Bool).
- `Type 1 (Sort 2)`: Polymorphic types quantifying over `Type 0`.

4. The Curry-Howard Isomorphism

Propositions as Types

- The architectural pillar of interactive theorem proving.
- A profound mathematical isomorphism between constructive logic and dependent programming.
- **Mathematical Proposition** $P \iff$ **Type** T
- **Formal Proof of** $P \iff$ **Term** t **inhabiting Type** T
- Writing a formal proof in Lean is structurally identical to writing a functional program!

Logic-to-Type Dictionary

Constructive Logic	**Dependent Type Theory**
Proposition (P)	Type (T)
Proof of P	Term t inhabiting Type T
Implication ($P \implies Q$)	Function Type ($A \to B$)
Universal Quant. ($\forall x, P(x)$)	Dependent Product (Π type)
Existential Quant. ($\exists x, P(x)$)	Dependent Sum (Σ type)
Conjunction ($P \wedge Q$)	Product / Tuple Type ($A \times B$)
True / Tautology ($\top$)	Unit Type
False / Contradiction ($\bot$)	Empty Type

Proof Irrelevance

- Consider a mathematical proposition $p : Prop$.
- If $t_1, t_2 : p$ represent two entirely distinct proof terms asserting the truth of p, Lean fundamentally treats t_1 and t_2 as definitionally equal.
- **Proof Irrelevance** guarantees that proofs carry no excess computational baggage beyond the pure fact that the proposition is true, aiding massive performance scaling.

5. Lean's Pipeline Architecture

Separation of Concerns

Lean 4 achieves vast scalability by architecturally separating complex heuristic elaboration from the minimal, highly trusted type-checking kernel.

1. **Macro Expansion**: Reduces syntactic sugar to core constructs.
2. **The Elaborator**: Resource-intensive component that fills in profound gaps, infers implicit arguments, resolves type classes, and generates a fully saturated "pre-term".

The Trusted Kernel

- The fully elaborated term is submitted to the trusted C++ kernel.
- **The Kernel is absolutely minimalist**. It represents the final source of truth.
- It performs no automation, typeclass inference, or heuristic deduction.
- It solely and mechanically verifies that the term complies with the fundamental formation and computation rules of CIC.

6. Machinery of Reduction

Propositional vs. Definitional Equality

- **Propositional Equality**: An explicit mathematical statement (`Eq a b`). Requires algebraic rewrites via theorems to generate an explicit proof term. (e.g., $x + y = y + x$).

- **Definitional Equality**: A weaker but automated syntactic relationship. Two terms compute to the exact same normal form via Lean's algorithms. (e.g., $2 + 3$ is definitionally equal to 5).

Reduction Algorithms

To establish definitional equality autonomously, the kernel utilizes:

- β-**Reduction**: Core mechanism of lambda calculus. Substitutes variables in function applications.
- δ-**Reduction**: Mechanically unwinds and expands explicit definitions.
- ι-**Reduction**: Unfolds primitive recursors applied to inductive constructors (powers pattern matching execution).
- ζ-**Reduction**: Expands local `let` bindings.

7. Python Run-time vs Lean Static Proofs

Dynamic Interpreter Constraints

- In Python, `2 + 3 == 5` is interpreted temporally at execution.
- The VM mathematically creates a new object 5, evaluates $5 == 5$, and pushes a `True` boolean singleton.
- The statement evaluates to a runtime primitive boolean entirely detached from static source code safety.

The Propositional Interpretation in Lean

- Lean views `2 + 3 = 5` strictly as a static type declaration: `Eq Nat (Nat.add 2 3) 5`.
- Writing `rfl` provides the proof term `Eq.refl _`.
- The Kernel uses δ and ι rules to reduce `Nat.add 2 3` structurally to ‘Nat.succ (Nat.succ 3)‘, which happens to be exactly the integer 5.
- Types match, type-check succeeds. The truth is verified completely unconditionally at compile-time.

False Type Expressions

- In Python, `2 + 3 == 6` produces the boolean `False` at runtime to branch logic.
- In Lean, writing `2 + 3 = 6` successfully constructs a perfectly valid type: `Eq Nat 5 6`...
- **But it is inherently uninhabited**. Logical contradiction ($\bot$).
- You physically cannot apply a constructor to inhabit it, failing compilation instantly. Truth is forever bound to type inhabitation.

8. Advanced Capabilities

The Equation Compiler & Metaprogramming

- **Equation Compiler**: Safely translates human-readable recursive pattern matching syntax heavily down into primitive kernel eliminators. It enforces strict structural termination to maintain logic consistency.
- **Lean 4 Metaprogramming**: Exposes internal C++ data structures via Lean itself, allowing users to define perfectly optimized custom syntax and solvers natively without external tools.

Tactical Automation & The Future

- Lean heavily leans on advanced tactics like `simp` via discrimination trees.
- Beyond pure algorithms, machine learning integrations like **Aesop** use neural networks to predict statistically probable proof steps on Mathlib.
- The synergy between rigorous formal verification and statistical AI marks the transition into true automated mathematics.

Conclusion

- Lean is more than a theorem prover; it's a bridge fusing mathematics and software engineering directly onto the Calculus of Inductive Constructions.
- Moving past empirical runtime observation into mathematically impenetrable compile-time guarantees is exactly the future of computing.

Thank You

Questions?